RESOLUÇÃO COMENTADA

# A Casa Sempre Ganha

Do Launch Template ao Auto Scaling Group, com o porquê de cada decisão.

## 01 Antes de começar

**Região.** `us-east-1`, escolhida no topo do console **antes de criar qualquer coisa**, e nunca mais trocada. Recurso criado em região errada é recurso invisível e cobrado.

**Orçamento.** Billing → Budgets → Create budget → Cost budget → **US$ 5**, alerta em 50% e 100% para o e-mail. Este desafio **sai do Free Tier** (ALB mais 5 instâncias): o budget não é cerimônia, é o airbag.

**Par de chaves.** EC2 → Key pairs → Create key pair → `girobet.pem` → `chmod 400 girobet.pem`. Baixa uma vez; perdeu, não entra mais.

> ⚠️ **A conta do desafio.** Uma tarde de laboratório (4h) custa ~US$ 0,31. Um mês esquecido ligado custa ~US$ 55. A diferença entre os dois números é a seção 09 deste guia. Combine com o grupo: montou, provou, desmontou.

## 02 A rede: dois Security Groups (tarefa 1)

EC2 → Security Groups → Create, **duas vezes**:

**`girobet-alb-sg`**, o segurança da porta da frente:

| PORTA | ORIGEM | POR QUE ESSA ORIGEM |
|---|---|---|
| 80 | `0.0.0.0/0` | é o site público; o mundo inteiro gira a roleta |
| 3000 | ninguém | **não crie esta regra.** Ninguém fala com o ALB na 3000 |

**`girobet-maquinas-sg`**, o segurança do salão das máquinas:

| PORTA | ORIGEM | POR QUE ESSA ORIGEM |
|---|---|---|
| 3000 | **`girobet-alb-sg`** | só o balanceador entrega giros às máquinas |
| 22 | `SEU_IP/32` | só você entra por SSH (tarefa 6) |

**A decisão que vale a aula:** a origem da regra 3000 é **o outro Security Group**, não um IP. Os IPs das máquinas e do ALB mudam a cada ciclo do ASG; o *grupo* não muda nunca. Regra que aponta para grupo sobrevive a qualquer terminate. É assim que se desenha rede para infraestrutura descartável.

**A prova**, depois de tudo no ar: `curl http://IP_DA_INSTANCIA:3000/health` de fora **falha** (timeout); `curl http://DNS_DO_ALB/health` **responde**.

## 03 O Launch Template e o user-data (tarefa 2)

EC2 → Launch Templates → Create launch template:

- **Name**: `girobet-maquina`
- **AMI**: Ubuntu Server 24.04 LTS · **Instance type**: `t3.micro`
- **Key pair**: `girobet` · **Security group**: `girobet-maquinas-sg`
- **Advanced details → Metadata response hop limit:** `2` ← a decisão escondida do desafio, explicada abaixo
- **Advanced details → User data**:

```
#!/bin/bash
# As máquinas da GiroBet nascem prontas: ninguém entra via SSH para
# instalar nada. Este script roda UMA vez, no primeiro boot, como root.

apt-get update -y
apt-get install -y docker.io

# O container sobe perdedor (sem PREMIADA) e renasce sozinho com a máquina.
docker run -d --name roleta --restart unless-stopped -p 3000:3000 \
  ghcr.io/liga-de-computacao-em-nuvem-pucrs/desafio-02-bet-backend:latest
```

**As quatro decisões do user-data:**

1. `--restart unless-stopped`: se o Docker ou a instância reiniciarem, o container volta. O ASG cuida de máquina morta; o restart policy cuida de container morto. São camadas diferentes de auto-cura, e o desafio usa as duas.
2. **Nenhum** `PREMIADA` **aqui.** O user-data é o mesmo para as 5 máquinas (é um *template*): todas nascem perdedoras, e a premiada é promovida à mão na tarefa 6. Se o aluno colocar `PREMIADA=true` no template, as **cinco** pagam, e o "1 em 5" vira "5 em 5". Erro clássico esperado.
3. **Hop limit 2.** A resposta do serviço de metadados (IMDSv2) decrementa um contador a cada salto de rede. Padrão: 1. A bridge do Docker gasta um salto extra → a resposta morre antes de chegar ao processo no container → o app cai no fallback e o painel mostra hostname de container em vez de `i-0abc...`. Com 2, chega. Nada quebra funcionalmente no Desafio 02 (por isso a dica é sutil no enunciado), mas no 03 esse mesmo limite derruba as **credenciais** da role.
4. **Sem** `docker login`**.** A imagem é pública no GHCR; o pull anônimo funciona. Se aparecer `denied`: ou o pacote não está público, ou o endereço tem letra maiúscula (o GHCR só aceita minúsculas).

**Depurar boot:** `ssh -i girobet.pem ubuntu@IP` → `cat /var/log/cloud-init-output.log` (a saída do user-data inteira) e `docker ps`.

## 04 Target Group e health check (tarefa 3)

EC2 → Target Groups → Create target group:

- **Target type**: Instances · **Protocol**: HTTP · **Port**: `3000`
- **VPC**: a default · **Health check path**: `/health`
- Thresholds padrão (healthy 5 · unhealthy 2 · interval 30s) servem.

**Não registre alvo nenhum à mão**: quem registra e desregistra máquinas no grupo é o ASG (seção 06). O Target Group é a "lista de presença" viva que o ALB consulta a cada requisição.

**Por que o `/health` da roleta responde sempre 200:** health check de balanceador pergunta "você está viva?", não "você está perfeita?". Se a API devolvesse 503 por estar "incompleta", o ALB tiraria as 5 máquinas de rotação e devolveria 502 para todo mundo, e o desafio morreria no nascedouro. Compare com o `/health` do Dojo, que servia para *diagnóstico humano*: mesmo nome, contratos diferentes, e a diferença é a audiência.

## 05 O Application Load Balancer (tarefa 4)

EC2 → Load Balancers → Create → **Application Load Balancer**:

- **Name**: `girobet-alb` · **Scheme**: Internet-facing
- **Availability Zones**: marque **duas** (ex.: `us-east-1a` e `us-east-1b`)
- **Security group**: `girobet-alb-sg` (tire o default)
- **Listener**: HTTP 80 → forward para o target group da seção 04

**Por que duas AZs, se as máquinas podem estar numa?** O ALB não é uma caixa: são nós espalhados por AZs, e a AWS exige no mínimo duas para o próprio balanceador sobreviver à queda de um datacenter. É o primeiro contato do grupo com "alta disponibilidade é geografia".

Anote o **DNS name** (`girobet-alb-123456.us-east-1.elb.amazonaws.com`): ele entra no `config.js` (seção 08) e nos testes.

## 06 O Auto Scaling Group (tarefa 5)

EC2 → Auto Scaling Groups → Create:

- **Name**: `girobet-maquinas` · **Launch template**: `girobet-maquina`
- **AZs**: as mesmas duas do ALB
- **Attach to an existing load balancer** → o target group da seção 04
- **Desired: 5 · Min: 5 · Max: 5**
- (Opcional, recomendado) **Turn on Elastic Load Balancing health checks**: o ASG passa a repor também a máquina cujo *container* morreu, não só a máquina desligada.

Em ~3 minutos: 5 instâncias `running`, e o Target Group com 5 targets `healthy`. Se algum ficar `unhealthy`, é quase sempre o SG (a 3000 aceita o SG do ALB?) ou o container que não subiu (cloud-init-output.log).

**A prova do self-healing (critério da tarefa 5):** EC2 → Instances → selecione uma → Terminate. O site não cai (o ALB para de mandar giros para ela em segundos) e o ASG cria uma substituta sozinho. Deixe o grupo cronometrar.

## 07 A máquina premiada (tarefa 6)

Escolha **uma** instância, pegue o IP público dela e:

```
ssh -i girobet.pem ubuntu@IP_DA_ESCOLHIDA

sudo docker rm -f roleta
sudo docker run -d --name roleta --restart unless-stopped -p 3000:3000 \
  -e PREMIADA=true \
  ghcr.io/liga-de-computacao-em-nuvem-pucrs/desafio-02-bet-backend:latest

curl -s localhost:3000/health   # → "premiada": true
```

**A prova estatística**, em três tamanhos:

1. No site, **modo turbo** (20 giros) algumas vezes: ~20% de vitória, 5 identificadores no painel, coroa em uma máquina só.
2. Por `curl`, 25 giros:

   ```
   for i in $(seq 1 25); do curl -s -X POST http://DNS_DO_ALB/api/girar | grep -o '"resultado":"[a-z_]*
   # esperado: ~5 premiado, ~20 nao_premiado
   ```

3. A prova de verdade, com o **estádio** (vem no repositório):

   ```
   node estadio.mjs http://DNS_DO_ALB --torcedores 200 --duracao 30
   ```

O placar final traz a fatia de giros por máquina, a taxa de vitória, a latência e dois vereditos automáticos: "balanceado" (fatias quase iguais) e "exatamente 1 premiada". Na validação local deste material, contra um balanceador simulado com 5 réplicas: 91.627 giros em 8s (~11.500 giros/s), fatias de 20,0% em cada máquina com diferença de 0,0 ponto, e 20,0% de vitórias. É este print que vale como evidência das tarefas 05 e 06 no material do grupo.

**A conversa importante do desafio:** o `PREMIADA=true` mora **no container de uma instância específica**. É estado mutável em infraestrutura descartável, um pecado cometido de propósito. A aposta alta (seção 10) faz o grupo pagar por ele e entender onde esse estado deveria morar (tag + leitura via IMDS, um parameter store, um banco...).

## 08 O frontend no S3 (tarefa 7)

Sem modo SPA desta vez, que é página única. S3 → Create bucket ( `girobet-site-SEUNOME`, desmarcar Block all public access) → Properties → Static website hosting → Enable, index `index.html` → Permissions → bucket policy de leitura pública (a mesma do Desafio 01).

**A novidade: configurar SEM rebuildar.** No repositório, edite `frontend/dist/config.js` no clone do repositório (**o de `dist/`**, que é o que sobe; o de `public/` é só o modelo):

```
window.LIGA_CONFIG = { API_URL: 'http://girobet-alb-123456.us-east-1.elb.amazonaws.com' };
```

Sem barra no final, `http` (não https). Depois:

```
aws s3 sync frontend/dist/ s3://girobet-site-SEUNOME/
aws s3 cp frontend/dist/config.js s3://girobet-site-SEUNOME/config.js \
  --cache-control no-store
```

O `--cache-control no-store` evita o navegador prender a versão velha do config, que é o arquivo que mais muda no desafio. Compare com o Desafio 01: lá, trocar a URL exigia `npm run build` de novo (a URL era assada dentro do bundle); aqui o `index.html` carrega `config.js` **antes** do bundle e lê `window.LIGA_CONFIG` em runtime. Mesma necessidade, duas estratégias (build-time vs runtime), e agora o grupo conhece as duas.

Abra pela **URL de website** do bucket (http) e gire.

## 09 A estimativa de custos (tarefa 8)

Premissas: `us-east-1`, on-demand, preços de meados de 2026 (confirmar na Pricing Calculator na hora de montar o material).

| ITEM | QUANTIDADE | PREÇO UNITÁRIO | MÊS (730H) |
|---|---|---|---|
| ALB (fixo) | 730 h | US$ 0,0225/h | US$ 16,43 |
| ALB (LCU, uso de laboratório) | ~1 LCU | US$ 0,008/LCU-h | ~US$ 0 a 6 |
| EC2 t3.micro | 5 × 730 h | US$ 0,0104/h | US$ 37,96 |
| EBS 8 GB gp3 | 5 × 8 GB | US$ 0,08/GB-mês | US$ 3,20 |
| S3 + transferência | ~1 GB | | < US$ 0,10 |
| **Total esquecido ligado** | | | **~US$ 55 a 60** |
| **Total de uma tarde (4h)** | | | **~US$ 0,31** |

Enquadramento para o material do grupo: o Free Tier clássico cobre 750h de **uma** t3.micro; aqui são cinco, mais um ALB que não tem free tier nenhum. Contas novas (pós julho/2025) têm US$ 100 de crédito, que absorvem o laboratório com folga, mas o número que importa é o **custo por hora (~US$ 0,08)** versus o **custo por esquecimento (~US$ 55/mês)**.

**"Por que a casa sempre ganha"** (o fecho do material do aluno): cada giro custa 150 GiroCoins e o prêmio paga 500. Chance de 1/5 → valor esperado do jogador por giro = −150 + 0,2 × 500 = **−50 GC**. A banca lucra 50 GC por giro **em média, por arquitetura**: não precisa de sorte, precisa de volume. É exatamente o que o painel do site mostra depois de 100 giros. E é por isso que bet de verdade não é investimento.

## 10 A aposta alta (tarefas 9 e 10)

**Terminar a premiada.** Melhor ainda com plateia: deixe o `estadio.mjs` rodando ( `--duracao 60` ) enquanto executa. EC2 → Instances → a premiada → Terminate. O estádio narra os erros por alguns segundos, o ALB tira a máquina de rotação, e a taxa de vitória despenca para 0% ao vivo. Em minutos o ASG cria a substituta, que nasce do template, logo **perdedora**. Site no ar, prêmio extinto: taxa de vitória vai a 0%. A explicação que vale nota: o ASG recupera o que está **declarado** (o template); o `PREMIADA=true` era estado **imperativo**, aplicado à mão numa máquina que por definição é descartável. Estado que precisa sobreviver não mora em instância.

**Sticky sessions.** Target Group → Attributes → Stickiness → ON (1 dia). O ALB passa a gravar um cookie ( `AWSALB` ) e a devolver o **mesmo** cliente sempre à **mesma** máquina. Resultado no navegador do aluno: 0% **ou** 100% de vitória, conforme a máquina sorteada no primeiro giro pós-cookie, e o painel congela numa máquina só. A resposta "qual dos dois?" antes de testar: 4/5 de chance de ser 0%. Stickiness existe para aplicações com sessão em memória; aqui destrói a premissa estatística. Mesma feature, contextos opostos. **Desligar depois do teste.**

## 11 Encerrando

**A limpeza, na ordem certa** (a ordem é conteúdo neste desafio):

1. **Auto Scaling Group** primeiro: desired/min → 0, depois Delete (ele termina as 5). Quem terminar instâncias antes disso assiste o ASG recriá-las: o self-healing virando conta.
2. Load Balancer → Delete; depois o Target Group.
3. Launch Template → Delete.
4. Bucket do site: esvaziar, depois apagar.
5. Billing no dia seguinte: custo do dia ≈ zero.

| SINTOMA | CAUSA QUASE SEMPRE |
|---|---|
| Targets `unhealthy` | SG das máquinas sem a regra "3000 ← SG do ALB", ou container morto (cloud-init-output.log) |
| ALB responde 502/503 | nenhum target healthy (veja a linha de cima) |
| Painel mostra nome de container | hop limit 1 (Launch Template → Advanced → Metadata response hop limit 2) |
| `docker pull` → `denied` | pacote GHCR privado, ou maiúscula no endereço da imagem |
| Nunca ganha | nenhuma máquina com PREMIADA=true (confira o `/health` dela) |
| Ganha sempre / painel com 1 máquina | PREMIADA no user-data (as 5 pagam) ou stickiness ligado |
| `curl` direto na instância funciona | tarefa 1 incompleta: a 3000 deve aceitar só o SG do ALB |
| Instância terminada "ressuscita" | comportamento correto do ASG; zere o desired antes |
| Site sem dados | config.js em localhost, cache (faltou no-store), ou site aberto em https |

**O que separa o grupo que tira nota cheia:** justifica o SG-referencia-SG sem ler; explica por que o user-data não tem PREMIADA; prevê o resultado das sticky sessions antes de ligar; e a estimativa fecha com a matemática da banca escrita para o Jorjão entender.